

Evaluation of quadrat-related functions

Gabriel Arellano

2026-05-31

Contents

1	Goal	1
2	Load code	2
3	Map dependencies	4
4	Evaluation	5
4.1	Goal and structure of most functions	5
4.2	Neighboring-related functions	7
4.3	Other functions	8
4.4	Depending functions	8
5	Thinking forward	9
5.1	Only one way of identifying quadrats	9
5.2	Small pieces vs. larger outputs	10
5.3	From internal to global	10
5.4	From global to internal	11
5.5	Neighboring quadrats, etc.	11
5.6	Other dependents	11
5.7	Not urgent	12
6	Clean up	12

1 Goal

There is a set of functions in the CTFS / fgeo functionality that seems related to handling quadrats and coordinates in ForestGEO plots. Most of the 20 functions seem to translate from

one form (and resolution) of location into another. These are the types of location information in this type of datasets:

- gx and gy: global coordinates within the plot, relative to a (0, 0) corner
- lx and ly: local coordinates within the quadrat
- col and row: column and row within the plot
 - conceptually, just coarse x and y coordinates
 - in practice, could be named with letters, start at 00 or 01, etc.
- quadrat name, following certain conventions, typically derived from row and col
- hectares (rarely)
- quadrat index, numerical
- quadrats of different resolution than the default
 - typically smaller quadrats nested within larger quadrats
 - sometimes with specific naming that refers to location within larger quadrat (e.g. 5x5 m quadrats, useful for fieldwork and filling maps, within 20x20 quadrats used for data analyses and summaries)

Additionally, there are functions related to identifying neighbors, selecting quadrats, and other related tasks. Probably secondary in importance.

This is the evaluation of the functions in the CTFS `quadfunc.r` script and, possibly, their downstream dependencies.

2 Load code

We declare paths to the project (watched by git), load a specific version of the code, and auxiliary functions.

```
# This is my local path to the project (being watched by git)
# You may have to adapt this.
path <- normalizePath(system2("git", c("rev-parse", "--show-toplevel"), stdout = TRUE)[1])
knitr::opts_knit$set(root.dir = path) # global wd, instead of within-chunk wd as setwd()

# Source useful functionality:
source(paste0(path, "/evaluation/0-code-to-source.r"))

# For reproducibility, this file does code evaluation at a specific
# point in time. Otherwise it won't make sense as the project
# moves forward and the code gets updated.
```

```

# The following lines create a temporary file and gives its path.
# You have to clean that at the end of the file, otherwise
# you will accumulate temporary folders somewhere.

DATE = "2026-05-29"
commit = commit_from_date(path = path, date = DATE)
historical_path <- make_dir_to_point_in_history(path = path, commit = commit)

# Other functionality:
library(igraph, quietly = TRUE)

```

Warning: package 'igraph' was built under R version 4.4.3

Attaching package: 'igraph'

The following objects are masked from 'package:stats':

decompose, spectrum

The following object is masked from 'package:base':

union

This will source the necessary scripts; note that I am loading a specific version of them, by using the path to a specific commit (`historical_path`, linked to `commit = 8b1c13366dba84a757fb08ebd9cf5d813c0d1668`).

```

modules <- c("01_utilities", "02_biomass", "03_spatial", "04_abundance_diversity",
             "05_demography_growth", "06_topography_habitat", "07_mapping_plotting")

scripts <- unique(unlist(lapply(modules, function(m) {
  list.files(paste0(historical_path, "/", m, "/R/new"),
             full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
})))

# Source scripts, then new:
for(f in scripts) source(f)

```

3 Map dependencies

This maps the mutual dependencies among the functions:

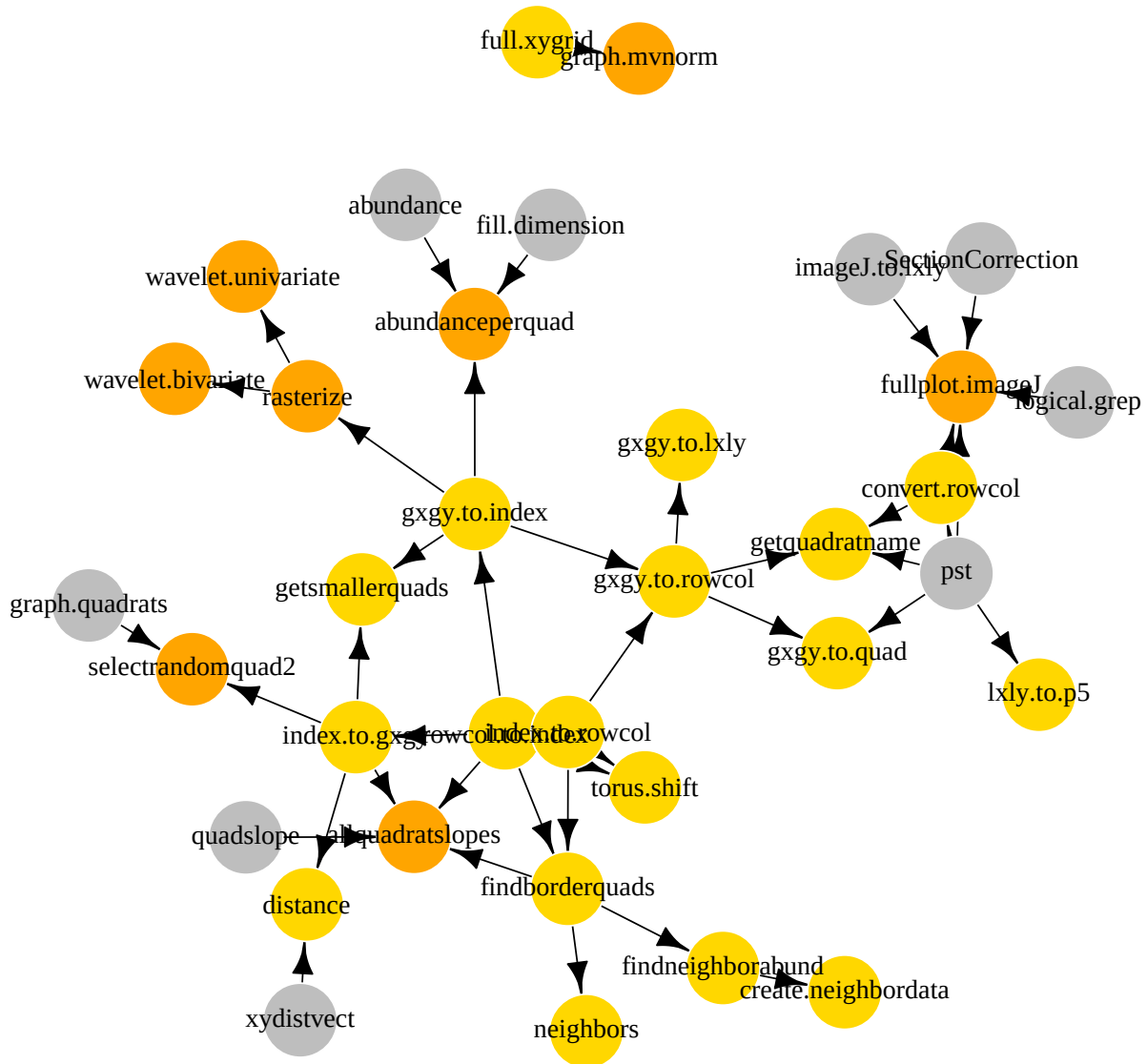
```
# All of the dependencies among all functions in the global environment:
edges <- get_all_dependencies()

# Identify the target subset of functions and their dependents:
where <- grep("quadfunc.r", scripts)
quadrat_functions <- functions_in_script(scripts[where])
down <- unique(unlist(lapply(quadrat_functions, function(q) downstream_dependents(q, edges))))
down <- down[!down %in% quadrat_functions]
target <- c(quadrat_functions, down)

# Graph representation:
keep <- edges$from %in% target | edges$to %in% target
edges <- edges[keep,,drop = FALSE]
g <- graph_from_data_frame(edges, directed = TRUE)

# Visualize
V(g)$color <- rep("gray", length(V(g)))
V(g)$color[V(g)$name %in% quadrat_functions] <- "gold"
V(g)$color[V(g)$name %in% down] <- "orange"

plot(g, layout = layout_fruchterman_reingold,
     vertex.label = V(g)$name,
     vertex.color = V(g)$color,
     vertex.frame.color = "white",
     vertex.label.color = "black",
     edge.color = "black")
```



4 Evaluation

This seems a relatively well-defined set of functions with surprisingly few dependents and only one direct dependency (`pst()` function, which is a CTFS version of `paste/collapse`).

4.1 Goal and structure of most functions

As said above, most functions seem to “translate” from one form and resolution of location information to another. The names of the functions inform about the possible ways of expressing

location:

- gx and gy: global coordinates
- lx and ly: local coordinates
- quadrat name
- quadrat index
- row and column (of quadrats)

There are somewhat cryptic mentions to smaller quadrats in the names of `getsmallerquads()` and `lxly.to.p5()` functions. No functions depend on them, so it is possible that they will be removed or substituted by more general ways of splitting into quadrats of arbitrary resolution.

It seems most of the difficult things require quadrat indexes, which is not something users will be familiar with, as the data tables typically store quadrat names, the fieldwork is done using quadrat names, etc. This, in itself, makes anything based on quadrat indexes difficult to de-bug or scrutinize, and thus it is a point of vulnerability.

There is an important naming convention for quadrats: CCRR, where CC are two digits for the column (location along x), and RR are two digits for the row (location along y). Both CC and RR could start at 00 or 01, depending on the site-level convention.

- the CCRR convention applies to BCI and other places, but it is not a universally accepted convention, so this is clearly a tricky point.
- the CCRR convention can be different for quadrats of different size, e.g. 5x5 m quadrats within 20x20 m quadrats only use one digit for (internal) column and one digit for (internal) row.
- the specification varies between CTFS functions, sometimes start = 0, sometimes start = “zero”.
- the specific CCRR convention should be trivial to guess from the complete data table, but other conventions for naming quadrats may not be that obvious.

The CCRR using numerical digits is a typical point of vulnerability in ForestGEO datasets, as it looks like a number, but it is not a number. Excel often goes from “0133” to “133”. This creates issues again and again. Condit code has warnings related to that, things like “Must rewrite if three digit quadrats”. The function `convert.rowcol()` adds initial zeros.

Interestingly, the `index.to.rowcol()` function returns numerical rows and columns, that always start with 1. So we can find datasets with row = 1, col = 1, and quadrat = 0000 in CCRR convention with start = 0.

There are two systems to uniquely identify quadrats within this code:

- quadrat name or code, following CCRR form

- `quadrat index`: numerical, one per quadrat. This is used internally to many of these functions, but does not directly affects important functionality (but check wavelet analysis, and imageJ-related code).

The functions apply to trees or quadrats, with quadrats being represented by their corners. For example:

- `quad.to.gxgy()`: returns global coordinates of the bottomleft coordinate of each quadrat, given a CCRR column-row naming convention, and a given gridsize (= quadrat size).
- `gxgy.to.quadrat()`: returns the quadrat membership of the trees, given their global coordinates.

There is a weird convention in Condit's code when coordinates are unknown, where `quadrat = 9999` instead of `quadrat = NA`. In contrast, `gxgy.to.index()` returns `quadrat index = NA` when there is not enough information to determine location.

4.2 Neighboring-related functions

There is a small set of functions related to identifying and comparing with neighboring quadrats... Their purpose is not clear, and it does not seems something people would use broadly. Possibly related to null models?:

- `create.neighbordata()`: risky and confusing. Takes as input an “abundanceperquad” object, which follows certain conventions (species in rows, data sorted by quadrat index). Clear candidate for removal. Nothing depends on this. The input is probably the output of the `abundanceperquad()` function, but the dependency is implicit, not explicit. It uses `findneighborabund()` (next).
- `findneighborabund()`: takes similar input, but called “abundvect” now. It finds neighboring quadrats using `findborderquads()` (next), and then copy-pastes abundances from the abundance vector.
- `findborderquads()`: calculates indexes of neighboring quadrats, for a given quadrat index. Probably useful, although using this just to calculate species abundances seems like a narrow application.
- `neighbors()`: very bad name, super general word, for an extremely narrow application. This function takes a vector of absence/presence for all quadrats, and returns proportion of presences among the neighboring quadrats. It is difficult to understand why this has become part of the broadly distributed CTFS package.
- `torus.shift()`: Creates a torus-shifted quadrat topographic dataset. This can be safely removed if a standard torus translation (or constrained, for that matter) is integrated into the pack of functions. Any quadrat-based map can be translated just by translating the corners, then going back to (null) quadrat codes, if needed.

Although not directly connected, there is an interesting function `distance()` that calculates the distances between quadrats. It only wraps from quadrat indexes to coordinates, and then applies `xydistvect()` function in `utilities > geometry`. Using distances seems a more general approach to identify neighboring quadrats, generalizable to $k = 4$, $k = 8$, and so on, using standard k -NN routines.

4.3 Other functions

`getsmallerquads()`: this is a dead-end function, nothing depends on it. It returns small quadrats within larger quadrats, given two resolutions. The smaller resolution will be adjusted internally. It seems a rather arbitrary sub-division of quadrats, instead of directly dividing the entire plot (or any point dataset, for that matter) into quadrat membership at any resolution.

4.4 Depending functions

There are different parts of CTFS / fgeo code that depend on the quadrat-related functions. So, if anything changes, the depending code must be adjusted accordingly (assuming it will be kept).

First, they relate with `fullplot.imageJ()`. That function is part of a practical routing to go from paper maps to digital coordinates. Still waiting to understand whether this is needed now that people work with tablets and all that.

Second, `gxgy.to.index()` and `index.to.gxgy()` relate to, apparently, rather trivial functions to pick quadrats or summarize abundance by quadrat: `abundanceperquad()` and `selectrandomquad2()`. Clearly, these tasks don't need dedicated functions. The user must be able to summarize abundances by any combination of things, e.g. [quadrat x species] or [family x size class x habitat], or whatever. Something similar can be said for `selectrandomquad2()`.

- Note: `selectrandomquad2()` does **not** pick a random quadrat among the available quadrats. Instead, it returns the coordinates of a square within the plot of a given size. It uses `index.to.gxgy()` internally as a trick, assuming 1x1 m grid. It seems something useful to do random visualizations of how the forest looks like at local scales, using `graph.quadrats()` internally. Will be reviewed later, but clearly weak candidate to remain as it is.

Third, `gxgy.to.index()` is called by functions that run the wavelet analysis applied to Forest-GEO datasets. This is probably something work isolating and developing by itself. Ideally, it should be generalize and integrated with other spatial null models. In any case, it does not seem a desirable or necessary dependency.

Fourth, there are links to a topographical function: `allquadratslopes()`.

Finally, there is an isolated pair of functions that should be examined together:

- `full.xygrid()` (in the `quadfunc.r` script) is equal to `base::expand.grid()`.
- `graph.mvnorm()` (in `utilities > distributions`) apparently is just used to graph contours on multivariate normal distributions). Maybe something to move to the mapping + plotting module.

5 Thinking forward

Clearly, the only piece of information really needed is just the global coordinates, $\{gx, gy\}$. That is what should be kept in a good, easy-to-use data table.

However:

- that is not always the case
- during data digitization the input can be local coordinates and quadrats (or sub-quadrats), so we still need functionality to end with global coordinates.
- other tables (maps) are often expressed as $\{\text{quadrat}, x\}$ pairs, for example $\{\text{quadrat}, \text{habitat}\}$, instead of spatially explicit information. Because this sub-optimal practice is widespread, main data tables must contain, in general, quadrat names or codes.

Here I am paying attention to code, not data conventions or good practices. The [5x5 m within 20x20 m quadrats] convention is something that ForestGEO users must be able to navigate with the new code. That means respect towards `lxly.to.p5()` functionality.

The new code has to be useful in these two scenarios:

- The user has internal coordinates and quadrats (e.g. from fieldwork) but not global coordinates
- The user has global coordinates: they should be able to derive anything from that.

5.1 Only one way of identifying quadrats

The CTFS code uses two ways of identifying quadrats: quadrat code or name (CCRR convention), and quadrat index (numerical, consecutive). The quadrat index is used often but mostly internally. Users are not familiar with that. It's not used for field work. It's rarely part of the distribution of the dataset.

In general: quadrat-level info can be linked to *any* quadrat-level unique ID. There is no need for more than one quadrat-level unique ID to link data with maps, etc.

It is possible that the users store maps or quadrat-level information using different quadrat-level unique IDs. We need ways of helping users figure out the location of those quadrats,

allowing flexibility in quadrat ID conventions. Otherwise they won't be able to link datasets with each other.

However, for the new code, it is better if it can work based on *any* way of identifying quadrats uniquely, regardless of whether that is a numerical code, or alpha-numerical, or whatever.

The main bottleneck, perhaps not solvable, would be to declare those quadrat-IDing conventions for a generality of cases. CCRR and consecutive numerical can be useful defaults or guesses.

5.2 Small pieces vs. larger outputs

The old code handles small pieces of information, maybe it's faster that way. But the different pieces of information work together: local coordinates don't make global sense without quadrat membership, and so on.

The idea of **global sense** may be important. Perhaps the output of all functions must be sufficient for it to make global sense. (This will be limited to the input, probably, as we do not want to enforce a specific definition of "global" beyond "everything that you included").

Thing to consider:

- if input are coordinates, the output must contain coordinates too
- moving quadrat IDs around is not safe without objects that specify their location: either the user have them, or we must create them right away. Given clear quadrat IDing convention, it should be easy to get (xmax, ymax, xmin, ymin), which is preferred over left-bottom corner or centroid or anything ambiguous.

Maybe accumulative outputs are safer than itemized outputs? For example, if input is global coordinates of trees, the output can be global coordinates of points + quadrat membership + local coordinates of points *and* also the quadrat-level table with (xmin, ymin, xmax, ymax) and attributes describing quadrat IDing convention.

5.3 From internal to global

Given quadrat-level information, sufficient to know quadrat-level location, and internal coordinates, one can get global coordinates.

The routine is the same for the whole plot, or for coordinates within large quadrats given subquadrats and within-subquadrat-level coordinates.

5.4 From global to internal

From global coordinates, one should be able to get anything.

A “split into quadrats of side = x” function in the script will be useful, in general:

- if applied to the entire plot, and fits well within 20 m, then it will do what `getsmallerquads()` does
- if not, then it’s just two classifications into non-nested quadrats
 - some resolution would leave unused borders, remainders
 - some resolutions will fit exactly, we can anticipate them
 - or we can find the closest resolution that will fit exactly, with a warning + attributes
- the same routing can be applied to local or global coordinates, so this could also do what `lxly.to.p5()` does.

Anytime a set of points is split into quadrats of a given resolution, the corresponding within-quadrat coordinates must be attached.

5.5 Neighboring quadrats, etc.

The neighbor-related functions, according to the “functions cannot be examples” principle, will be probably substituted by:

- A function to indentify neighboring quadrats
 - possibly constrained (e.g. only neighbors of the same habitat, or same community type, etc., using arbitrary grouping)
 - possibly through a “distance between quadrats” intermediate step, so we can generalize to k=4, k=8, etc., using k-NN routines.
- Tutorials or examples: abundances, presence/absence, basal area, ...

As happens with the neighborhood analyses (of points): neighboring quadrats are just pieces of forest, subsets of trees, and there are many things that could be done with that, not just a couple.

5.6 Other dependents

Although not fully developed now, modify the following functions so they don’t break:

- `fullplot.imageJ()`
- `allquadratslopes()`

- `selectrandomquad2()`
- `rasterize()`, part of the wavelet analysis
- `abundanceperquad()`, but consider direct removal
- `graph.mvnorm()`: depends on `full.xygrid() = base::expand.grid()` (apparently)

5.7 Not urgent

- quadrats are just polygons... Perhaps there are potential gains if we facilitate a polygon representation for quadrats. Probably enough by giving (xmin, ymin, xmax, ymax).

6 Clean up

```
# Remove the temporary folder we created to
# access the project at a specific point in history:
knitr::opts_knit$set(root.dir = path)
if(dir.exists(historical_path))
  remove_dir_to_point_in_history(path, historical_path)

dir.exists(historical_path) # should be FALSE now
```

```
[1] FALSE
```